

PROJECTE FINAL

Escola intel·ligent

Fer una escola més agradable



Seba Parvu & Iker Miranda
21/05/2026
Proje. Inter.
Dani Soldevila

ÍNDEX

Objectiu i context del projecte.....	3
Descripció del problema.....	3
Solució proposada.....	3
Explicació tècnica.....	5
Fritzing.....	7
Diagrama de flux del codi sencer.....	8
Issues.....	10
Video demostració.....	28

Objectiu i context del projecte

L'objectiu principal del projecte és crear un sistema domòtic ambiental intel·ligent basat en una ESP32 capaç de monitoritzar les condicions de l'entorn i actuar automàticament segons els valors detectats. El sistema combina sensors, actuadors i una interfície web per controlar i visualitzar dades en temps real.

El projecte està enfocat a simular un sistema de control ambiental com els que es poden utilitzar en hivernacles, habitacions intel·ligents o espais automatitzats. Per això, es treballa amb sensors de temperatura i llum, així com amb diferents actuadors que permeten reaccionar davant els canvis ambientals.

A més, el sistema incorpora connexió WiFi i una pàgina web accessible des de qualsevol dispositiu de la mateixa xarxa, facilitant la supervisió i el control remot del sistema.

Descripció del problema

En molts espais interiors o sistemes domòtics és necessari controlar automàticament factors ambientals com la temperatura o la il·luminació. Si aquests valors no es regulen correctament, es poden generar problemes de confort, consum energètic o funcionament inadequat dels equips.

El projecte busca solucionar aquesta necessitat desenvolupant un sistema capaç de:

- Detectar la temperatura ambiental amb el sensor LM35.
- Mesurar la llum ambiental amb el sensor TEMA6000.
- Mostrar les dades en temps real en una interfície web.
- Activar actuadors automàticament segons les condicions detectades.
- Permetre un mode manual i un mode automàtic de funcionament.

D'aquesta manera, el sistema és capaç de reaccionar de forma intel·ligent davant els canvis de l'entorn sense necessitat d'intervenció constant de l'usuari.

Solució proposada

La solució proposada consisteix en el desenvolupament d'un sistema intel·ligent basat en una ESP32 que integra sensors, actuadors i una interfície web per monitoritzar i controlar les condicions ambientals de manera automàtica.

El sistema és capaç de llegir dades de temperatura i llum en temps real, processar aquesta informació i actuar segons unes regles definides prèviament. Mitjançant la connexió WiFi, l'usuari pot visualitzar l'estat del sistema des d'una pàgina web accessible des de qualsevol dispositiu connectat a la mateixa xarxa.

A més, el projecte incorpora diferents actuadors, com un servo motor, LEDs indicadors i una tira LED intel·ligent, que permeten simular funcions de ventilació i il·luminació automàtica. També s'implementa un mode manual per donar més control a l'usuari sobre el funcionament del sistema.

1. Connexió ESP32 i comunicació Serial

S'utilitza una placa ESP32 com a controlador principal del sistema. La ESP32 es connecta a la xarxa WiFi i inicialitza la comunicació Serial a 115200 bauds per facilitar el debug i el seguiment del funcionament del sistema durant el desenvolupament.

2. Servidor web integrat

La ESP32 incorpora un servidor web bàsic que permet accedir a una pàgina des del navegador. Aquesta web funciona com a interfície de monitoratge del sistema.

3. Interfície web

Es crea una estructura HTML amb un disseny senzill i clar per mostrar les dades ambientals i l'estat dels actuadors. La interfície permet veure la informació en temps real de forma visual i accessible.

4. Sensor de temperatura LM35

El sensor LM35 s'encarrega de llegir la temperatura ambiental. La ESP32 processa aquests valors i els utilitza tant per mostrar-los a la web com per activar el sistema de ventilació automàtica.

5. Sensor de llum TEMA6000

El sensor TEMA6000 permet detectar la quantitat de llum ambiental. Els valors llegits serveixen per controlar la il·luminació del sistema mitjançant la tira LED intel·ligent.

6. Visualització de dades a la web

Les dades dels sensors es mostren en temps real a la pàgina web. Això permet supervisar fàcilment la temperatura i el nivell de llum des de qualsevol dispositiu connectat a la xarxa.

7. LED indicador d'estat

S'utilitza un LED indicador per mostrar l'estat general del sistema. Per exemple:

- Estat normal → LED verd.
- Estat d'alerta o temperatura elevada → LED vermell.

Això facilita identificar ràpidament el funcionament del sistema.

8. Control del servo de ventilació

S'implementa un servo motor que simula un sistema de ventilació automàtica. Quan la temperatura supera un valor determinat, el servo es mou per representar l'obertura d'una ventilació.

9. Control de la tira LED intel·ligent

La tira LED ajusta la seva intensitat segons la llum ambiental detectada. Quan hi ha poca llum, els LEDs augmenten la il·luminació; quan hi ha molta llum, la intensitat disminueix.

10. Lògica automàtica del sistema

La ESP32 executa una lògica automàtica basada en regles:

- Temperatura alta → activació del servo de ventilació.
- Poca llum → augment de la il·luminació LED.
- Situacions d'alerta → activació del LED indicador.

Aquestes regles permeten automatitzar completament el comportament del sistema.

11. Mode manual i automàtic

S'afegeix un push button per canviar entre mode manual i mode automàtic. Això permet a l'usuari decidir si el sistema ha d'actuar sol o si vol controlar manualment els actuadors.

12. Integració completa del sistema

Finalment, tots els components (sensors, actuadors i servidor web) s'integren en un únic sistema funcional capaç de monitoritzar i controlar l'entorn de manera intel·ligent.

Explicació tècnica

Issue #	Nom	Descripció
#1	Connexió ESP32 i Serial	Inicialitzar ESP32, configurar WiFi i activar Serial a 115200 per debug.
#2	Servidor web bàsic	Crear una pàgina web accessible des de la ESP32.
#3	Disseny interfície web	Estructura HTML inicial amb layout de dades ambientals.

#4	Lectura sensor temperatura (LM35)	Llegir i mostrar temperatura en temps real.
#5	Lectura sensor llum (TEMT6000)	Obtenir valors de llum ambiental i validar funcionament.
#6	Visualització dades a web	Mostrar temperatura i llum en la interfície web.
#7	Control LED indicador	Activar LED segons estat del sistema (normal/alerta).
#8	Control servo ventilació	Moure servo segons temperatura per simular ventilació.
#9	Control tira LED intel·ligent	Ajustar il·luminació segons nivell de llum ambiental.
#10	Lògica automàtica del sistema	Definir regles: temperatura + llum → actuadors automàtics.
#11	Botó mode manual	Afegir push button per canviar automàtic/manual.
#12	Integració completa sistema	Unir sensors, actuadors i web en un sistema funcional.
#13	Vídeo demostració final	Gravar el sistema funcionant i pujar-lo a Drive o README.
#14	Documentació tècnica	Redactar memòria dins /doc/ (es pot fer en paral·lel).

Fritzing

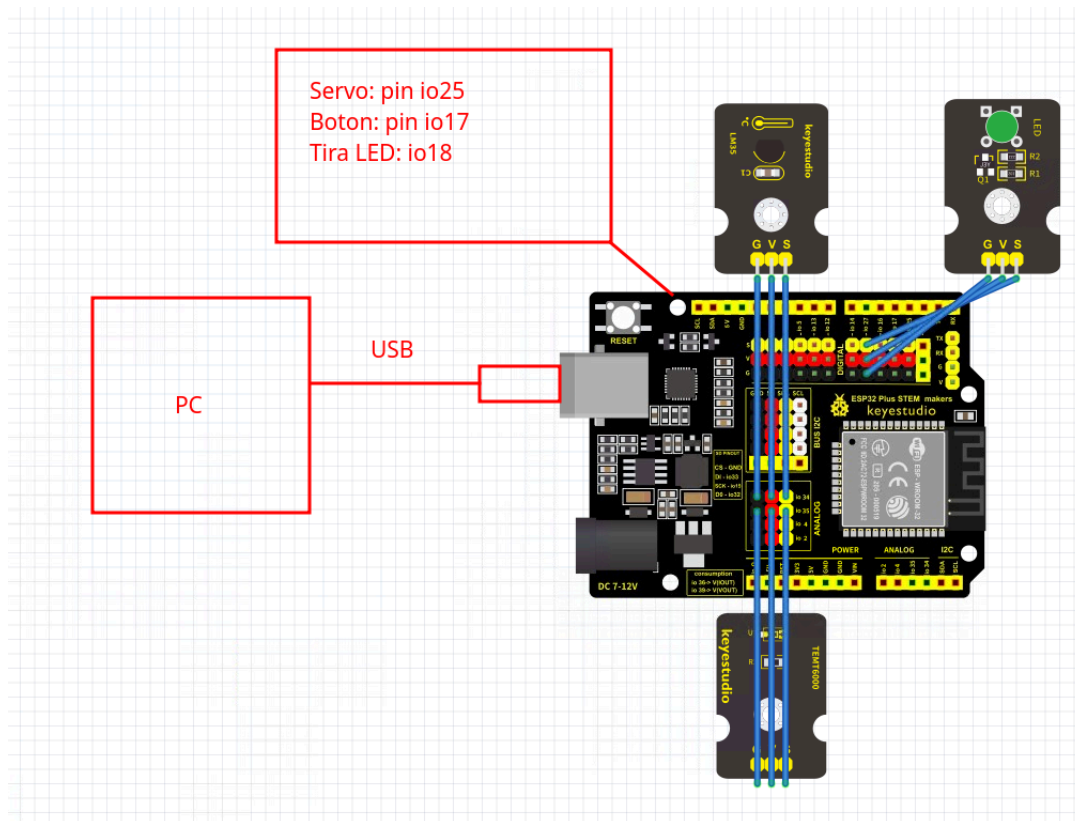
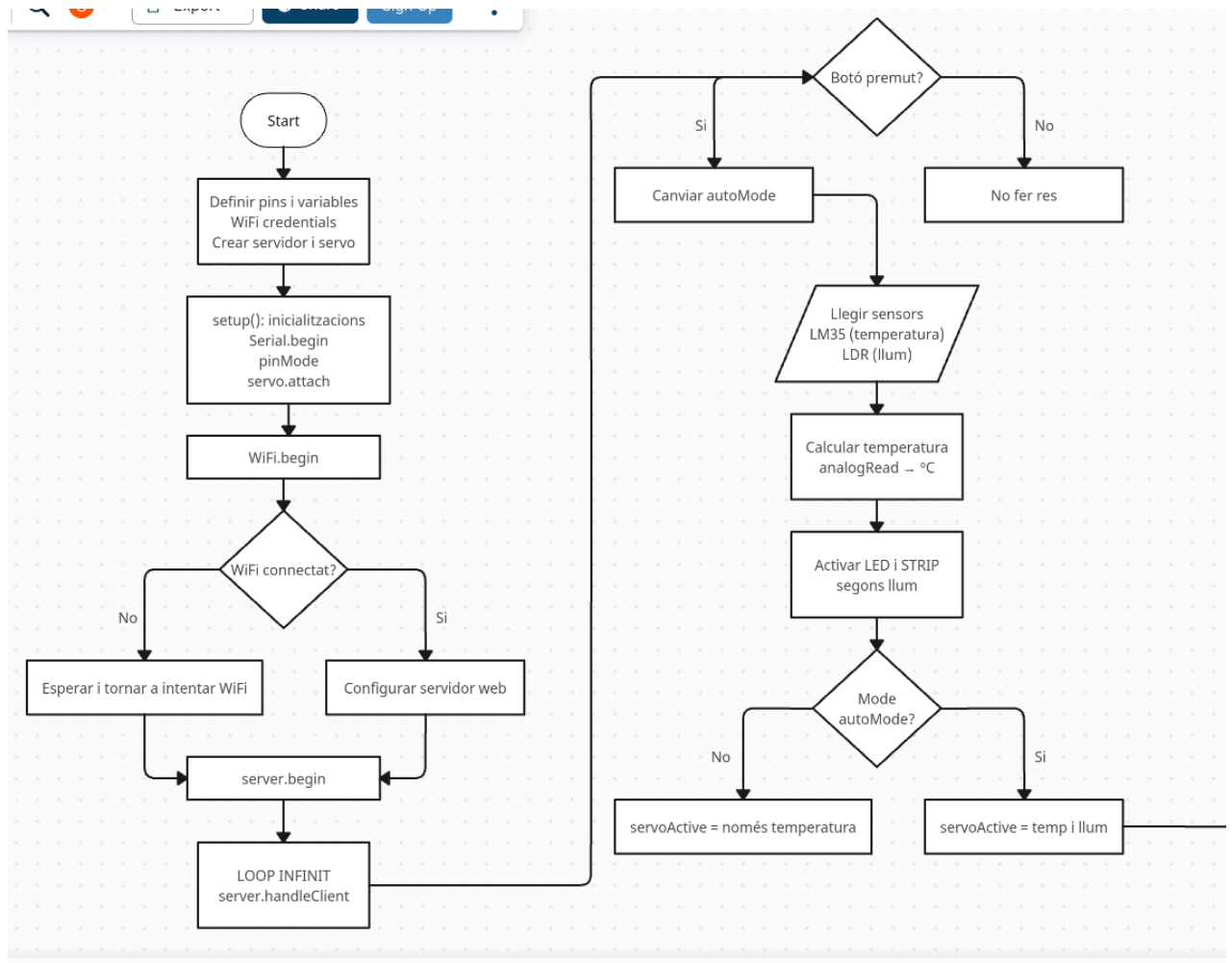
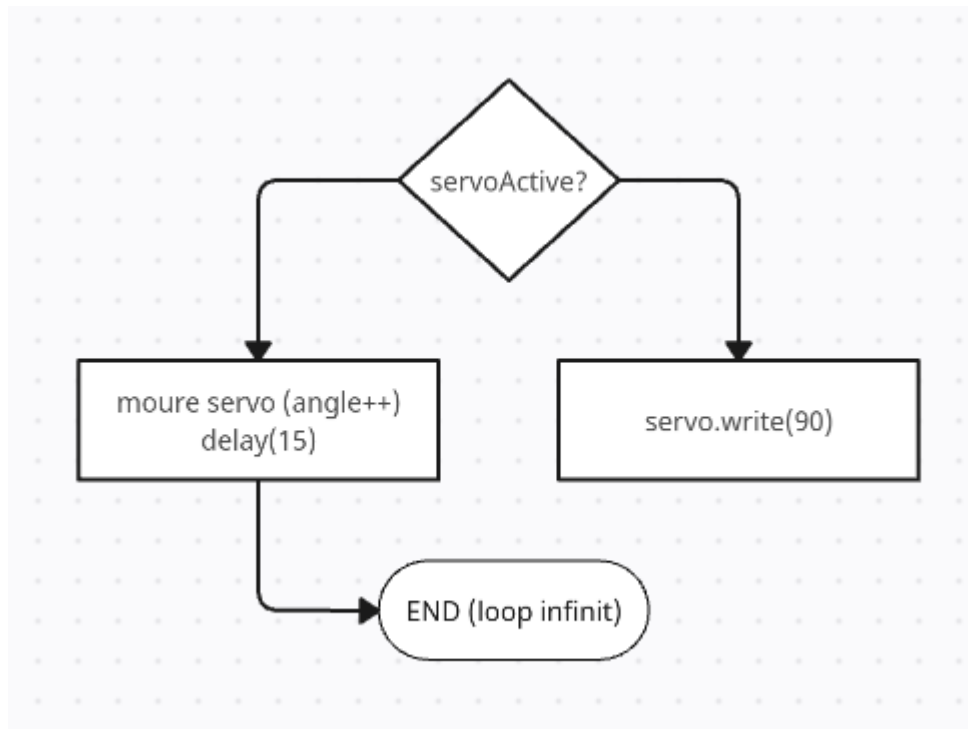


Diagrama de flux del codi sencer



<https://app.createely.com/d/5sCQrRS9m6l/edit>



<https://app.creately.com/d/keVLEcwO51a/edit>

Això es la continuació del d'adalt només que la pàgina no ens deixava afegir més coses ja que havíem que pagar

Issues

Issue 1:

```
#include <WiFi.h>

const char* ssid = "ProjectesSMX";
const char* password = "SI123456";

void setup() {
    Serial.begin(115200);

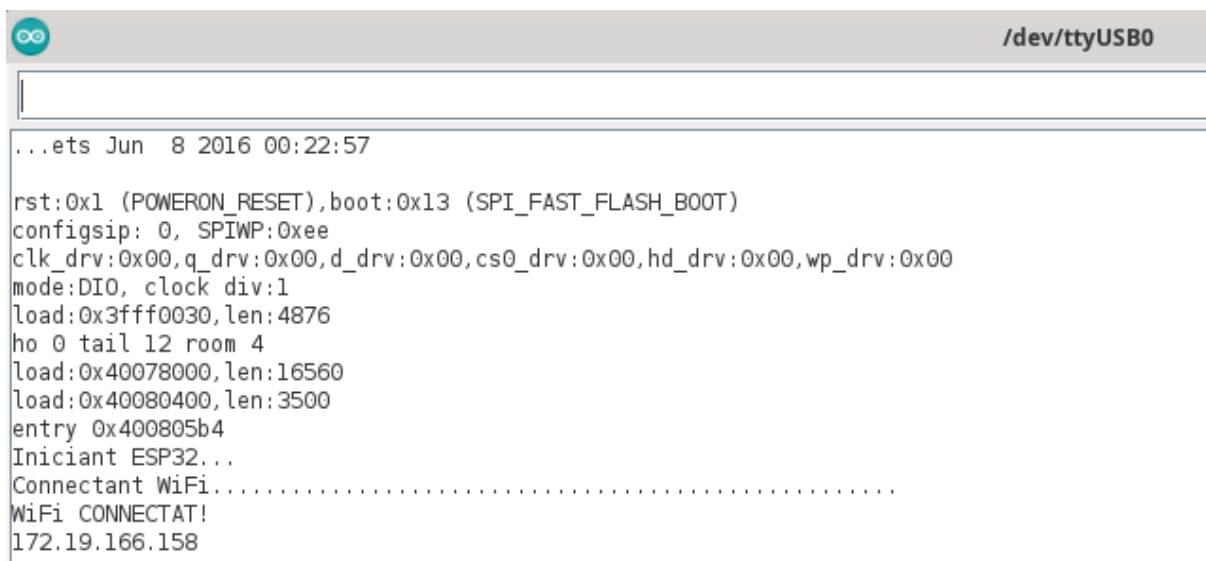
    Serial.println("Iniciant ESP32...");

    WiFi.begin(ssid, password);
    Serial.print("Connectant WiFi");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nWiFi CONNECTAT!");
    Serial.println(WiFi.localIP());
}

void loop() {}
```



```
...ets Jun 8 2016 00:22:57
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4876
ho 0 tail 12 room 4
load:0x40078000,len:16560
load:0x40080400,len:3500
entry 0x400805b4
Iniciant ESP32...
Connectant WiFi.....
WiFi CONNECTAT!
172.19.166.158
```

En tots el codis sortirà això. Et dona la ip per connectar-te a la web

Issue 2:

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "ProjectesSMX";
const char* password = "SI123456";

WebServer server(80);

void handleRoot() {
  server.send(200, "text/html", "<h1>Escola Intel·ligent activa</h1>");
}

void setup() {
  Serial.begin(115200);

  Serial.println("Iniciant ESP32...");

  WiFi.begin(ssid, password);
  Serial.print("Connectant WiFi");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi CONNECTAT!");
  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.begin();
}

void loop() {
  server.handleClient();
}
```

Issue 3:

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "ProjectesSMX";
const char* password = "SI123456";

WebServer server(80);

void handleRoot() {
  String page = "<html><body>";
  page += "<h1>Escola Intel·ligent</h1>";
  page += "<h2>Dades Ambientals</h2>";
  page += "<p>Temperatura: --</p>";
  page += "<p>Llum: --</p>";
  page += "</body></html>";

  server.send(200, "text/html", page);
}

void setup() {
  Serial.begin(115200);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }

  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.begin();
}

void loop() {
  server.handleClient();
}
```

Issue 4:

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "ProjectesSMX";
const char* password = "SI123456";

WebServer server(80);

#define LM35 34

float tempC;

void handleRoot() {
  String page = "<html><body>";
  page += "<h1>Escola Intel·ligent</h1>";
  page += "<p>Temperatura: " + String(tempC) + " C</p>";
  page += "</body></html>";

  server.send(200, "text/html", page);
}

void setup() {
  Serial.begin(115200);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }

  server.on("/", handleRoot);
  server.begin();
}

void loop() {
  tempC = (analogRead(LM35) * 3.3 / 4095.0) * 100;

  server.handleClient();
}
```

Issue 5:

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "ProjectesSMX";
const char* password = "SI123456";

WebServer server(80);

#define LM35 34
#define LIGHT 35

float tempC;
int lightValue;

void handleRoot() {
  String page = "<html><body>";
  page += "<h1>Escola Intel·ligent</h1>";
  page += "<p>Temperatura: " + String(tempC) + " C</p>";
  page += "<p>Llum: " + String(lightValue) + "</p>";
  page += "</body></html>";

  server.send(200, "text/html", page);
}

void setup() {
  Serial.begin(115200);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }

  server.on("/", handleRoot);
  server.begin();
}

void loop() {
  tempC = (analogRead(LM35) * 3.3 / 4095.0) * 100;
  lightValue = analogRead(LIGHT);

  server.handleClient();
}
```

Issue 6:

```
#include <WiFi.h>
#include <WebServer.h>

#define LM35 34
#define LIGHT 35

const char* ssid = "ProjectesSMX";
const char* password = "SI123456";

WebServer server(80);

float tempC;
int lightValue;

void handleRoot() {

    String page = "<html><body>";

    page += "<h1>Escola Intel·ligent</h1>";
    page += "<h2>Dades ambientals</h2>";

    page += "<p>Temperatura: " + String(tempC) + "</p>";
    page += "<p>Llum: " + String(lightValue) + "</p>";

    page += "</body></html>";

    server.send(200, "text/html", page);
}

void setup() {
    Serial.begin(115200);

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) delay(500);

    server.on("/", handleRoot);
    server.begin();
}

void loop() {
    tempC = (analogRead(LM35) * 3.3 / 4095.0) * 100;
    lightValue = analogRead(LIGHT);

    server.handleClient();
}
```

Issue 7:

```
#include <WiFi.h>
#include <WebServer.h>

#define LM35 34
#define LIGHT 35
#define LED 27

const char* ssid = "ProjectesSMX";
const char* password = "SI123456";

WebServer server(80);

float tempC;
int lightValue;

void handleRoot() {

    String page =
    "<html><body style='font-family:Arial;'>"
    "<h1 style='font-size:32px;'>Escola Intel·ligent</h1>"
    "<h2 style='font-size:20px; color:black;'>Dades ambientals</h2>"
    "<p style='font-size:16px;'>Temperatura: " + String(tempC) + " °C</p>"
    "<p style='font-size:16px;'>Llum: " + String(lightValue) + "</p>"
    "</body></html>";

    server.send(200, "text/html", page);
}

void setup() {
    Serial.begin(115200);

    pinMode(LED, OUTPUT);

    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) delay(500);

    server.on("/", handleRoot);
    server.begin();
}

void loop() {
    tempC = (analogRead(LM35) * 3.3 / 4095.0) * 100;
    lightValue = analogRead(LIGHT);

    digitalWrite(LED, lightValue < 1500);

    server.handleClient();
}
```


Issue 8:

```
#include <WiFi.h>
#include <WebServer.h>
#include <ESP32Servo.h>

#define LM35 34
#define LIGHT 35
#define LED 27

const char* ssid = "ProjectesSMX";
const char* password = "SI123456";

WebServer server(80);
Servo servo;

float tempC;
int lightValue;

int angle = 90;
int stepDir = 10;

void handleRoot() {

    String page =
    "<html><body style='font-family: Arial; '>"
    "<h1>Escola Intel·ligent</h1>"
    "<h2>Dades ambientals</h2>"
    "<p>Temperatura: " + String(tempC) + " °C</p>"
    "<p>Llum: " + String(lightValue) + "</p>"
    "</body></html>";

    server.send(200, "text/html", page);
}

void setup() {
    Serial.begin(115200);

    pinMode(LED, OUTPUT);
    servo.attach(25);

    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) delay(500);

    Serial.println("WiFi CONNECTAT");
    Serial.println(WiFi.localIP());

    server.on("/", handleRoot);
    server.begin();
}

void loop() {

    tempC = (analogRead(LM35) * 3.3 / 4095.0) * 100;
    lightValue = analogRead(LIGHT);

    digitalWrite(LED, lightValue < 1500);

    if (tempC > 13.5) {
        angle += stepDir;

        if (angle >= 180 || angle <= 0) {
            stepDir = -stepDir;
        }

        servo.write(angle);
        delay(30);
    }

    server.handleClient();
}
```

Issue 9:

```
#include <WiFi.h>
#include <WebServer.h>
#include <ESP32Servo.h>

// 📌 PINOS
#define LM35 34
#define LIGHT 35
#define LED 27
#define STRIP 14
#define SERVO_PIN 25

// 📶 WIFI
const char* ssid = "ProjectesSMX";
const char* password = "SI123456";

// 🌐 SERVER
WebServer server(80);
Servo servo;

// 📊 VARIABLES
float tempC;
int lightValue;

// ⚙️ SERVO
int angle = 90;
int stepDir = 10;

// 🌐 PÀGINA WEB
void handleRoot() {

    String page =
    "<html><body style='font-family:Arial;'>"
    "<h1>Escola Intel·ligent</h1>"
    "<h2>Dades ambientals</h2>"
    "<p>Temperatura: " + String(tempC) + " °C</p>"
    "<p>Llum: " + String(lightValue) + " (0-4095)</p>"
    "</body></html>";

    server.send(200, "text/html", page);
}

// ⚙️ SETUP
void setup() {
    Serial.begin(115200);

    pinMode(LED, OUTPUT);
    pinMode(STRIP, OUTPUT);

    servo.attach(SERVO_PIN);

    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
    }

    Serial.println("WiFi CONNECTAT");
    Serial.print("IP: ");
    Serial.println(WiFi.localIP());

    server.on("/", handleRoot);
    server.begin();
}
```

```

// 🔄 LOOP
void loop() {

    server.handleClient();

    // 📡 SENSORS
    tempC = (analogRead(LM35) * 3.3 / 4095.0) * 100;
    lightValue = analogRead(LIGHT);

    // 💡 LLUM CONTROL (FIXAT 1500)
    digitalWrite(LED, lightValue < 1500);
    digitalWrite(STRIP, lightValue < 1500);

    // 🌀 SERVO (ACTIU SI CALOR)
    if (tempC > 13.5) {

        angle += stepDir;

        if (angle >= 180 || angle <= 0) {
            stepDir = -stepDir;
        }

        servo.write(angle);
        delay(30);
    }
}

```

Issue 10:

```
#include <WiFi.h>
#include <WebServer.h>
#include <ESP32Servo.h>

#define LM35 34
#define LIGHT 35
#define LED 27
#define STRIP 14
#define SERVO_PIN 25

const char* ssid = "ProjectesSMX";
const char* password = "SI123456";

WebServer server(80);
Servo servo;

float tempC;
int lightValue;

int angle = 90;
int stepDir = 10;

void handleRoot() {

    String page =
    "<html><body style='font-family:Arial;'>"
    "<h1>Escola Intel·ligent</h1>"
    "<h2>Dades ambientals</h2>"
    "<p>Temperatura: " + String(tempC) + " °C</p>"
    "<p>Llum: " + String(lightValue) + "</p>"
    "</body></html>";

    server.send(200, "text/html", page);
}

void setup() {
    Serial.begin(115200);

    pinMode(LED, OUTPUT);
    pinMode(STRIP, OUTPUT);

    servo.attach(SERVO_PIN);

    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) delay(500);

    Serial.println("WiFi CONNECTAT");
    Serial.println(WiFi.localIP());

    server.on("/", handleRoot);
    server.begin();
}
```

```

void loop() {

    server.handleClient();

    tempC = (analogRead(LM35) * 3.3 / 4095.0) * 100;
    lightValue = analogRead(LIGHT);

    // ☞ LLUM
    digitalWrite(LED, lightValue < 1500);
    digitalWrite(STRIP, lightValue < 1500);

    // ☞ LÒGICA AUTOMÀTICA COMBINADA
    if (tempC > 13.5 && lightValue < 3000) {

        angle += stepDir;

        if (angle >= 180 || angle <= 0) {
            stepDir = -stepDir;
        }

        servo.write(angle);
        delay(30);

    } else {
        servo.write(90);
    }
}

```

Issue 11:

```
#include <WiFi.h>
#include <WebServer.h>
#include <ESP32Servo.h>

#define LM35 34
#define LIGHT 35
#define LED 27
#define STRIP 14
#define SERVO_PIN 25
#define BUTTON 17

const char* ssid = "ProjectesSMX";
const char* password = "SI123456";

WebServer server(80);
Servo servo;

float tempC;
int lightValue;

bool autoMode = true;

int angle = 0;
int stepDir = 3; // \ més suau

unsigned long lastButtonPress = 0;

void handleRoot() {

  String page =
    "<html><body style='font-family:Arial;'>"
    "<h1>Escola Intel·ligent</h1>"
    "<h2>Dades ambientals</h2>"
    "<p>🌡 Temperatura: " + String(tempC) + " °C</p>"
    "<p>💡 Llum: " + String(lightValue) + "</p>"
    "<h3>Mode: " + String(autoMode ? "AUTO 🏠" : "MANUAL 🚗") + "</h3>"
    "</body></html>";

  server.send(200, "text/html", page);
}

void setup() {
  Serial.begin(115200);

  pinMode(LED, OUTPUT);
  pinMode(STRIP, OUTPUT);
  pinMode(BUTTON, INPUT_PULLUP);

  servo.attach(SERVO_PIN);
  servo.write(90);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);

  Serial.println("WiFi CONNECTAT");
  Serial.print("IP: ");
  Serial.println(WiFi.localIP());

  server.on("/", handleRoot);
  server.begin();
}
```

```

void loop() {

    server.handleClient();

    // 🗲 BOTÓ
    if (digitalRead(BUTTON) == LOW && millis() - lastButtonPress > 400) {
        autoMode = !autoMode;
        lastButtonPress = millis();
    }

    // 📏 SENSORS
    tempC = (analogRead(LM35) * 3.3 / 4095.0) * 100;
    lightValue = analogRead(LIGHT);

    // 💡 LLUM
    digitalWrite(LED, lightValue < 1500);
    digitalWrite(STRIP, lightValue < 1500);

    // 🌀 CONDICIÓN SERVO
    bool servoActive;

    if (autoMode) {
        servoActive = (tempC > 13.5 && lightValue < 1500);
    } else {
        servoActive = (tempC > 13.5);
    }

    // 🔄 BUCLE 0° ↔ 180°
    if (servoActive) {

        angle += stepDir;

        if (angle >= 180 || angle <= 0) {
            stepDir = -stepDir;
        }

        servo.write(angle);
        delay(15);

    } else {
        servo.write(90);
    }
}

```

Issue 12:

```
#include <WiFi.h>
#include <WebServer.h>
#include <ESP32Servo.h>

#define LM35 34
#define LIGHT 35
#define LED 27
#define STRIP 14
#define SERVO_PIN 25
#define BUTTON 17

const char* ssid = "ProjectesSMX";
const char* password = "SI123456";

WebServer server(80);
Servo servo;

float tempC = 0;
int lightValue = 0;

bool autoMode = true;

int angle = 0;
int stepDir = 3;

unsigned long lastButtonPress = 0;

void handleRoot() {

    // 📡 ESTATS
    bool tempAlert = tempC > 13.5;
    bool lightAlert = lightValue < 1500;

    bool servoActive;

    if (autoMode) {
        servoActive = (tempC > 13.5 && lightValue < 1500);
    } else {
        servoActive = (tempC > 13.5);
    }

    String modeText = autoMode ? "AUTO" : "MANUAL";

    String tempColor = tempAlert ? "red" : "green";
    String lightColor = lightAlert ? "red" : "green";

    String servoStatus = servoActive ? "ACTIU" : "PARAT";
    String ledStatus = lightAlert ? "ENCES" : "APAGATS";

    String servoColor = servoActive ? "green" : "gray";

    String page = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<meta http-equiv="refresh" content="2">

<title>Escola Intel·ligent</title>

<style>
```



```

body{
  font-family: Arial;
  background:#f2f2f2;
  text-align:center;
  margin-top:40px;
}

.container{
  width:350px;
  margin:auto;
  background:white;
  padding:25px;
  border-radius:15px;
  box-shadow:0px 0px 10px gray;
}

h1{
  color:#222;
}

h2{
  color:black;
  font-size:22px;
}

.card{
  margin:15px 0;
  padding:12px;
  border-radius:10px;
  background:#eeeeee;
  font-size:20px;
}

.status{
  font-weight:bold;
}

</style>
</head>

<body>

<div class="container">

<h1>Escola Intel·ligent</h1>

<h2>Dades Ambientals</h2>
)rawliteral";

page += "<div class='card'>🌡 Temperatura: <span style='color:' + tempColor + '>' + String(tempC) + " °C</span></div>";
page += "<div class='card'>💡 Llum: <span style='color:' + lightColor + '>' + String(lightValue) + "</span></div>";
page += "<div class='card'>🔌 Mode: <span class='status'>" + modeText + "</span></div>";
page += "<div class='card'>🌀 Servo: <span style='color:' + servoColor + '>' + servoStatus + "</span></div>";
page += "<div class='card'>💡 LED/Tira LED: <span class='status'>" + ledStatus + "</span></div>";
page += R"rawliteral(

```

```

</div>
</body>
</html>
)rawliteral";

    server.send(200, "text/html", page);
}

void setup() {

    Serial.begin(115200);

    pinMode(LED, OUTPUT);
    pinMode(STRIP, OUTPUT);
    pinMode(BUTTON, INPUT_PULLUP);

    servo.attach(SERVO_PIN);
    servo.write(90);

    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
    }

    Serial.println("WiFi CONNECTAT");
    Serial.print("IP WEB: ");
    Serial.println(WiFi.localIP());

    server.on("/", handleRoot);

    server.begin();
}

void loop() {

    server.handleClient();

    // ☉ BOTÓ AUTO / MANUAL
    if (digitalRead(BUTTON) == LOW &&
        millis() - lastButtonPress > 400) {

        autoMode = !autoMode;

        lastButtonPress = millis();
    }

    // 📖 LECTURA SENSORS
    tempC = (analogRead(LM35) * 3.3 / 4095.0) * 100;

    lightValue = analogRead(LIGHT);

    // ♀ LEDS LLUM
    digitalWrite(LED, lightValue < 1500);

    digitalWrite(STRIP, lightValue < 1500);

    // 🌀 LÒGICA SERVO
    bool servoActive;

```

```

// 🏠 AUTO
if (autoMode) {

    servoActive =
        (tempC > 13.5 && lightValue < 1500);

}

// 🏠 MANUAL
else {

    servoActive =
        (tempC > 13.5);

}

// ⚙️ MOVIMENT SERVO
if (servoActive) {

    angle += stepDir;

    if (angle >= 180 || angle <= 0) {
        stepDir = -stepDir;
    }

    servo.write(angle);

    delay(15);

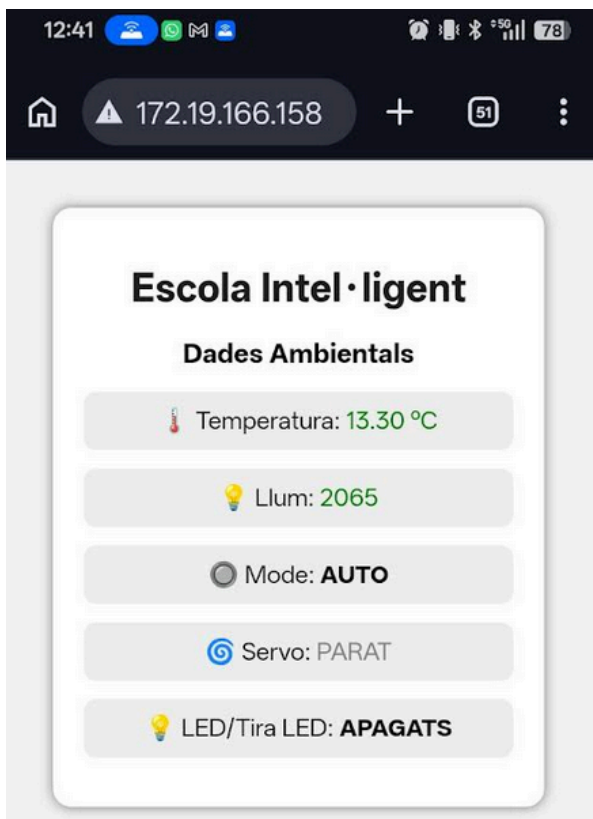
}

else {

    servo.write(90);

}
}

```



El que es veu a la web

- Temperatura actual
- Nivell de llum
- Mode actual (AUTO o MANUAL)
- Estat del servo
- Estat dels LEDs

El programa llegeix dues dades principals:

- la temperatura del sensor LM35
- la llum ambiental del sensor d'il·luminació

A partir d'aquestes dades, el sistema decideix què fer:

- Si hi ha poca llum (menys de 1500), s'encenen el LED i la tira LED.
- Si la temperatura supera els 13,5 °C, el servo es pot activar.

Després, el funcionament depèn del mode seleccionat:

Mode AUTO - El servo només s'activa quan:

- la temperatura és alta
- i hi ha poca llum

Mode MANUAL - El servo funciona sempre que:

- la temperatura sigui alta

El botó serveix per canviar entre el mode AUTO i el mode MANUAL.

Video demostració

https://docs.google.com/videos/d/1NSVnsFITQmHili4-s5hLxS_MZBY_T61wO_js9xl3Lms/edit?scene=id.p#scene=id.p